Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №3

«Графи. Способи представлення графів. Остовні дерева. Пошук найкоротших шляхів»

Виконав:

студент групи ІО-45

Унятицький А. Д.

Залікова книжка №4523

Перевірив Пономаренко А.М.

Київ – 2025

Лабораторна робота № 3

Тема: «Графи. Способи представлення графів. Остовні дерева. Пошук найкоротших шляхів».

Мета роботи: Вивчення властивостей графів, способів їх представлення та основних алгоритмів на графах.

Завдання: створити програму, яка реалізує один з алгоритмів на графах.

Теоретичні основи:

Алгоритм Дейкстри знаходить відстань від однієї вершини (дамо їй номер 0) до всіх інших, використовуючи кількість операцій порядку $O(n^2)$. Всі ваги ребер у графі додатні. Базовою структурою даних для цього алгоритму є список g[0..n-1][0..n-1], кожен елемент якого g[i][j] містить вагу ребра, що з'єднує вершину i з вершиною j у випадку, якщо таке ребро існує, або inf, якщо відповідного ребра в графі не існує. На кожній ітерації позначають певну підмножину вершин. Задамо список з логічними змінними mark [0 ... n - 1]. Індекс цього списку відповідає номеру вершини. Значення кожного елементу списку дорівнює «True», якщо вершина позначена і дорівнює «False», якщо вершина не позначена. Задамо також список d [0 ... n - 1]такого ж розміру. В цьому списку для кожної вершини буде зберігатися довжина найкоротшого шляху, що проходить тільки по помічених вершинах, як по «пересадочних» . Також підтримується інваріант того, що для помічених вершин довжина, яка зазначена в d, і є відповідь. Спочатку позначена тільки вершина 0, а g [0][j] дорівнює x, якщо вершина 0 з'єднана ребром вагою x з вершиною j, або inf, якщо їх не з'єднує ребро, і дорівнює 0, якщо j = 0. На кожній ітерації ми знаходимо вершину, з найменшим значенням в d серед непомічених, нехай це вершина v. Тоді значення d [v] є відповіддю для v.

Завдання: 4

4	1. Розглянути основні принципи пошуку найкоротших шляхів у графі. Написати програму за алгоритмом Дейкстри для пошуку найкоротших шляхів у графі.

№	Опис варіанта
	2. За правилом, наданим викладачем, сформувати матрицю ваг $C = \left[c_{i,j} \right]$ графа. 3. Представити граф, заданий матрицею ваг C, у графічній формі та виділити найкоротший шлях між заданими викладачем вершинами, сформований за алгоритмом Дейкстри.

Код:

```
from tkinter import *
from random import *
import networkx as nx
import matplotlib.pyplot as plt
from PIL import Image, ImageTk


class DijkstraApp:
    def __init__(self):
        self.window = Tk()
        self.window.title('Лабораторна робота 3')
        self.window.config(bg='#87CEEB')
        self.window.geometry('1200x700+100+10')

        self.matrix = []
        self.ribs = []

        self.setup_ui()

    def setup_ui(self):
        self.graph_frame = Frame(self.window, bg="#4682B4", height=400, width=460)
        self.graph_frame.grid(row=2, column=1, rowspan=5, padx=10, pady=10)

        self.matrix_frame = Frame(self.window, bg="#4682B4", height=200, width=300)
        self.matrix_frame.grid(row=2, column=2, rowspan=2, padx=10, pady=10)

        info_frame = Frame(self.window, bg="#B0C4DE")
        info_frame.grid(row=0, column=0, columnspan=3, sticky="ew", padx=10, pady=5)

        Label(info_frame, text="Унятицький Антон Дмитрович", font="Arial 12", bg="#B0C4DE",
fg="#191970").pack(side=LEFT, padx=5)
        NZK = 4523
```

```
N = 23
I = (NZK % 10) + 1
Label(info_frame, text=f"Група: ІО-45 | № у списку: {N} | Варіант: {I}",
    font="Arial 12", bg="#B0C4DE", fg="#191970").pack(side=LEFT)

input_frame = Frame(self.window, bg="#87CEEB")
input_frame.grid(row=2, column=0, rowspan=4, padx=10, pady=10, sticky="ns")

Label(input_frame, text="Введіть кількість вершин:", font="Arial 12", bg="#87CEEB",
fg="#191970").pack(pady=5)
self.vertices = Entry(input_frame, width=10, font="Arial 12")
self.vertices.pack(pady=5)

Label(input_frame, text="Введіть максимальну вагу:", font="Arial 12", bg="#87CEEB",
fg="#191970").pack(pady=5)
self.max_weight = Entry(input_frame, width=10, font="Arial 12")
self.max_weight.pack(pady=5)

Label(input_frame, text="Початкова вершина:", font="Arial 12", bg="#87CEEB",
fg="#191970").pack(pady=5)
self.start_vertex = Entry(input_frame, width=10, font="Arial 12")
self.start_vertex.pack(pady=5)

Label(input_frame, text="Кінцева вершина:", font="Arial 12", bg="#87CEEB",
fg="#191970").pack(pady=5)
self.end_vertex = Entry(input_frame, width=10, font="Arial 12")
self.end_vertex.pack(pady=5)

Button(self.window, text="Згенерувати граф і матрицю", font="Arial 12",
    bg="#4169E1", fg="white", command=self.generate_graph).grid(row=8, column=1,
pady=10)
```

```
        Label(self.window, text="Граф:", font="Arial 12", bg="#87CEEB",
fg="#191970").grid(row=1, column=1, pady=5)
        Label(self.window, text="Матриця ваг:", font="Arial 12", bg="#87CEEB",
fg="#191970").grid(row=1, column=2, pady=5)


    def dijkstra(self, matrix, start, end, vertex):
        distances = [float('inf')] * vertex
        distances[start-1] = 0
        visited = [False] * vertex
        previous = [None] * vertex

        for _ in range(vertex):
            min_dist = float('inf')
            current = -1
            for i in range(vertex):
                if not visited[i] and distances[i] < min_dist:
                    min_dist = distances[i]
                    current = i
            if current == -1:
                break
            visited[current] = True
            for j in range(vertex):
                if not visited[j] and matrix[current][j] != float('inf'):
                    new_dist = distances[current] + matrix[current][j]
                    if new_dist < distances[j]:
                        distances[j] = new_dist
                        previous[j] = current

        path = []
        current = end - 1
        while current is not None:
            path.append(current + 1)
```

```
        current = previous[current]
    return path[::-1], distances[end-1]

def generate_table(self, vertex, weight, start, end):
    self.ribs.clear()
    d = 0
    number_of_ribs = choice(range(2, ((vertex - 2) * vertex)))
    while d < number_of_ribs:
        i = choice(range(1, vertex + 1))
        j = choice(range(1, vertex + 1))
        if i != j and [i, j] not in self.ribs and [j, i] not in self.ribs:
            self.ribs.append([i, j])
            d += 1

    print(f"Згенеровані ребра:\n {self.ribs}")
    self.matrix = [[float('inf')] * vertex for _ in range(vertex)]
    for i in range(vertex):
        self.matrix[i][i] = 0

    for k in self.ribs:
        weight_gen = choice(range(1, weight + 1))
        self.matrix[k[0]-1][k[1]-1] = weight_gen
        self.matrix[k[1]-1][k[0]-1] = weight_gen

    print("Матриця ваг:")
    for r in self.matrix:
        print(r)

    path, distance = self.dijkstra(self.matrix, start, end, vertex)
    print(f"Найкоротший шлях від {start} до {end}: {path}")
    print(f"Відстань: {distance}")
```

```
        for widget in self.matrix_frame.winfo_children():
            widget.destroy()

        for i in range(vertex + 1):
            for j in range(vertex + 1):
                pole = Entry(self.matrix_frame, font="Arial 12", width=5, justify='center', bd=2,
bg="#ADD8E6")
                value = "" if i == 0 and j == 0 else f"v{j}" if i == 0 else f"v{i}" if j == 0 else self.matrix[i-
1][j-1]
                pole.grid(row=i, column=j, padx=1, pady=1)
                pole.insert(END, value)

        return path

    def generate_graph(self):
        try:
            start = int(self.start_vertex.get())
            end = int(self.end_vertex.get())
            vertex = int(self.vertices.get())
            weight = int(self.max_weight.get())

            path = self.generate_table(vertex, weight, start, end)

            G = nx.Graph()
            G.add_edges_from([(i-1, j-1, {'weight': self.matrix[i-1][j-1]}) for i, j in self.ribs])

            pos = nx.spring_layout(G)
            fig, ax = plt.subplots(figsize=(8, 8))

            nx.draw_networkx_nodes(G, pos, ax=ax, node_color='#87CEFA', node_size=800)
            nx.draw_networkx_labels(G, pos, ax=ax, font_size=16, font_color="#191970")
```

```
            nx.draw_networkx_edges(G, pos, ax=ax, width=1.5, edge_color="#4682B4")
            nx.draw_networkx_edge_labels(G, pos,
                                         edge_labels={(i-1, j-1): self.matrix[i-1][j-1] for i, j in self.ribs},
                                         font_size=12, font_color="#191970")

            path_edges = [(path[i]-1, path[i+1]-1) for i in range(len(path)-1)]
            nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color='#FF4500', width=2)

            plt.savefig("graph.png", bbox_inches='tight')
            plt.close()

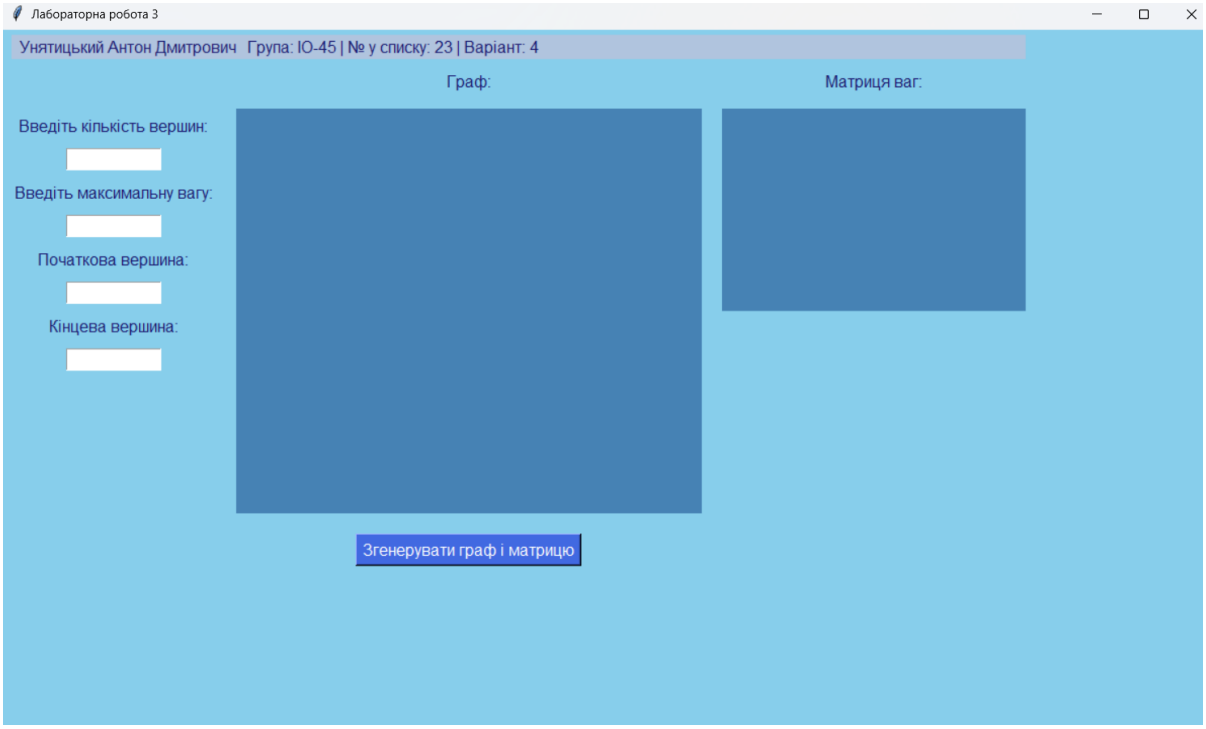
            image = Image.open("graph.png").resize((460, 460), Image.Resampling.LANCZOS)
            photo = ImageTk.PhotoImage(image)
            image_label = Label(self.graph_frame, image=photo)
            image_label.image = photo
            image_label.grid(row=0, column=0)

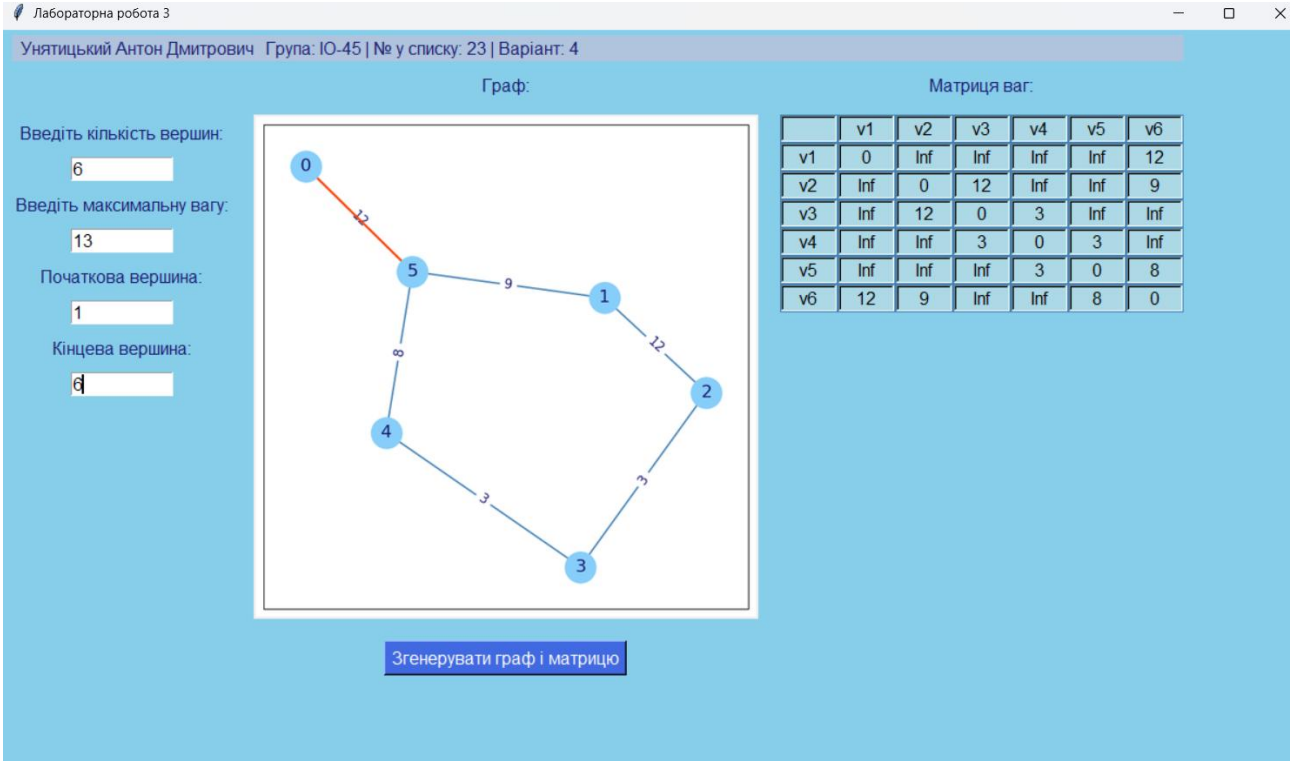
        except ValueError:
            print("Помилка: введіть коректні числові значення")

    def run(self):
        self.window.mainloop()

if __name__ == "__main__":
    app = DijkstraApp()
    app.run()
```

Скріни:





Висновок. На цій лабораторній роботі я вивчив що таке граф , способи задавання графів. Також розглянув алгоритм побудови Дейкстри й написав програму пошуку найкоротших шляхів у графі за цим алгоритмом. Згідно матриці найкоротших шляхів, наведеної в скриншоті, програма працює правильно.